

**Banco de Problemas - Fase 5 - Evaluación Final
POA**

Erika Yadira Hernandez Romero

Cedula: 1052397930

Tutor: Neider Duan Barbosa Castro

Fundamentos de programación

Código curso: 213022_972

**Universidad Nacional abierta y a distancia-UNAD
(ECBTI) Ingeniería de sistemas**

Mayo 2026

Introducción

Este trabajo fue desarrollado con el fin de desarrollar un problema práctico de la vida real mediante la programación de Python, aplicando los conceptos de matrices y funciones que nos pide la guía del curso. El ejercicio consiste en crear un sistema automático para auditar el inventario de una bodega, comparando el número de productos que tenemos guardados con el stock mínimo de productos que debe existir, para lograrlo realice una lista organizada (matriz) y desarrolle una lógica muy clara; y así cuando nos hagan falta algún producto nos marcara cuanto debemos de adquirir para así en un caso de la vida real podría ser que no nos faltaran los productos en una bodega de almacenamiento como un negocio o en la casa, con este trabajo logro demostrar la importancia de las herramientas digitales para una gestión de negocio, esto permite que, en un caso de la vida real. Un negocio o almacén no se quede desabastecido.

Objetivos

Objetivo general

- Desarrollar habilidades de pensamiento computacional y análisis de algoritmos mediante la identificación y corrección de errores en un código base de Python utilizando la inteligencia artificial como herramienta de apoyo pedagógico.

Objetivos específicos

- Identificar errores de sintaxis y lógica en un código, solucionar estos errores con apoyo de la inteligencia artificial
- Interactuar con herramientas de inteligencia artificial (copilot) mediante el uso de Prompts para obtener un diagnóstico exacto de los posibles errores en la programación del código.

Implementar procesos de mejora mediante evidencias visuales y argumentar de una manera crítica estas mejoras generadas por la IA

1.planteamiento del problema

Se requiere el desarrollo de una herramienta computacional automatizada que permita auditar el inventario de una bodega y decidir que artículos se necesitan ser reabastecidos. La información de los productos se encuentra estructurada dentro de una matriz que contiene el código del artículo, nombre, stock actual y Stock mínimo Requerido. Mediante la evaluación existencial el sistema debe determinar de forma exacta el déficit y consolidar una lista detallada de pedidos para optimizar el reabastecimiento.

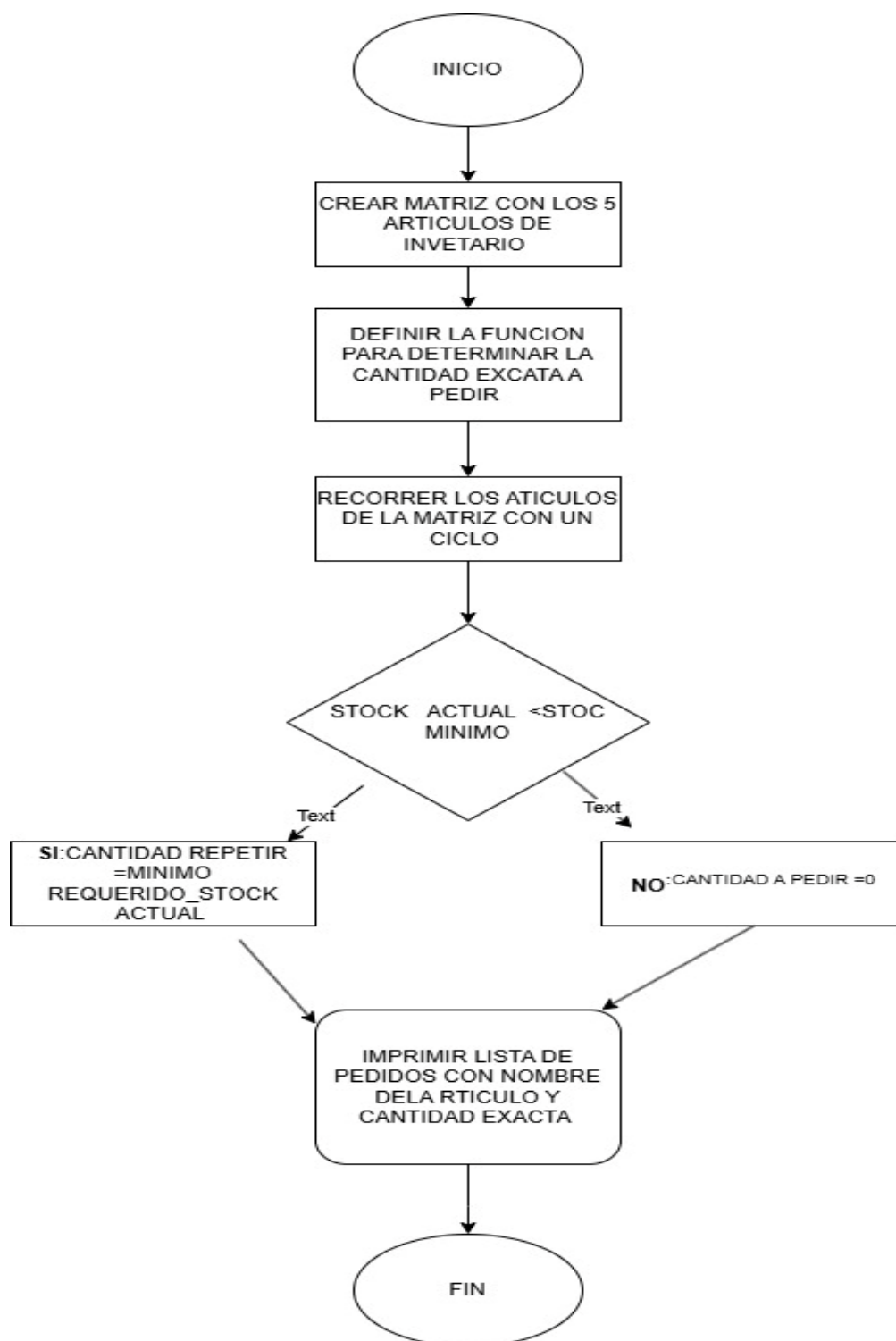
Tabla de requerimientos

ID	REQUERIMIENTO	DESCRIPCION	TIPO
RF01	Crear matriz de autoría	Construir una estructura de datos bidimensional que almacene un mínimo de 5 artículos con sus campos: código articulo nombre stock actual stock mínimo requerido	Funcional
RF02	Crear módulo de calculo	Implementar una función independiente orientada a procesar los datos de cada articulo y determinar matemáticamente la cantidad exacta a pedir	Funcional
RF03	Validar condición de reabastecimiento	Aplicar una estructura lógica condicional para verificar si el stock actual es menor al stock mínimo requerido activando el cálculo del déficit.	Funcional
RF05	Validar existencias suficientes	Controlar mediante lógica condicional que si el stock actual es mayor igual o	Funcional

		mínimo el valor de unidades a solicitar se asigne automáticamente en cero.	
RF06	Imprimir listas de pedidos	Generar una salida en pantalla limpia que actúe como reporte final. Mostrando exclusivamente el nombre del artículo y la cantidad exacta que debe ser solicitada.	Funcional
RF07	Lenguaje de programación	La solución del algoritmo técnico debe programarse obligatoriamente sobre el desarrollo del lenguaje Python	No Funcional
RF08	Uso de estructuras del curso	El código debe organizarse de forma modular incorporado funciones definidas colecciones de tipo arreglo (matrices y ciclos de repetición)	No Funcional
RF09	Legibilidad y documentación	El programa fuente debe contener comentarios claros que explique la lógica de negocio y variables fáciles de entender	No Funcional

TABLA DE ANALISIS DEL PROBLEMA

Elemento	descripción
Entrada	Datos de los artículos cargados en la matriz: Código Artículo, Nombre. Stock actual y Stock mínimo requerido.
Proceso	recorrer la matriz fila por fila inspeccionando cada articulo mediante una interacción For Invocar el módulo para evaluar stock $\text{actual} < \text{stock mínimo}$ Aplicar lógica de negocio, si es s menor calcular.
Salida	Impresión en la terminal de la lista de pedidos mostrando de forma exclusiva el nombre del articulo y la cantidad exacta que debe solicitar.

DIAGRAMA DE FLUJO

Problema 3

Problema 3: Se requiere una herramienta para auditar el inventario y decidir qué artículos necesitan ser reabastecidos. La información se encuentra en una matriz: [Código Artículo, Nombre, Stock Actual, Stock Mínimo Requerido].

Requisitos de Desarrollo

- **Matriz:** Crear una matriz con al menos 5 artículos.
- **Módulos:** Se requiere un módulo (función) para determinar la **cantidad exacta a pedir** para un artículo.

- Lógica de Negocio:

✓ Si el **Stock Actual es menor al Stock Mínimo**, la cantidad a pedir es la diferencia (Mínimo Requerido - Stock Actual).

✓ Si el **Stock Actual es suficiente** (mayor o igual al Mínimo), la cantidad a pedir es **cero**.

.- **Salida:** Imprimir una lista de pedidos que muestre el nombre del artículo y la cantidad exacta que debe ser solicitada.

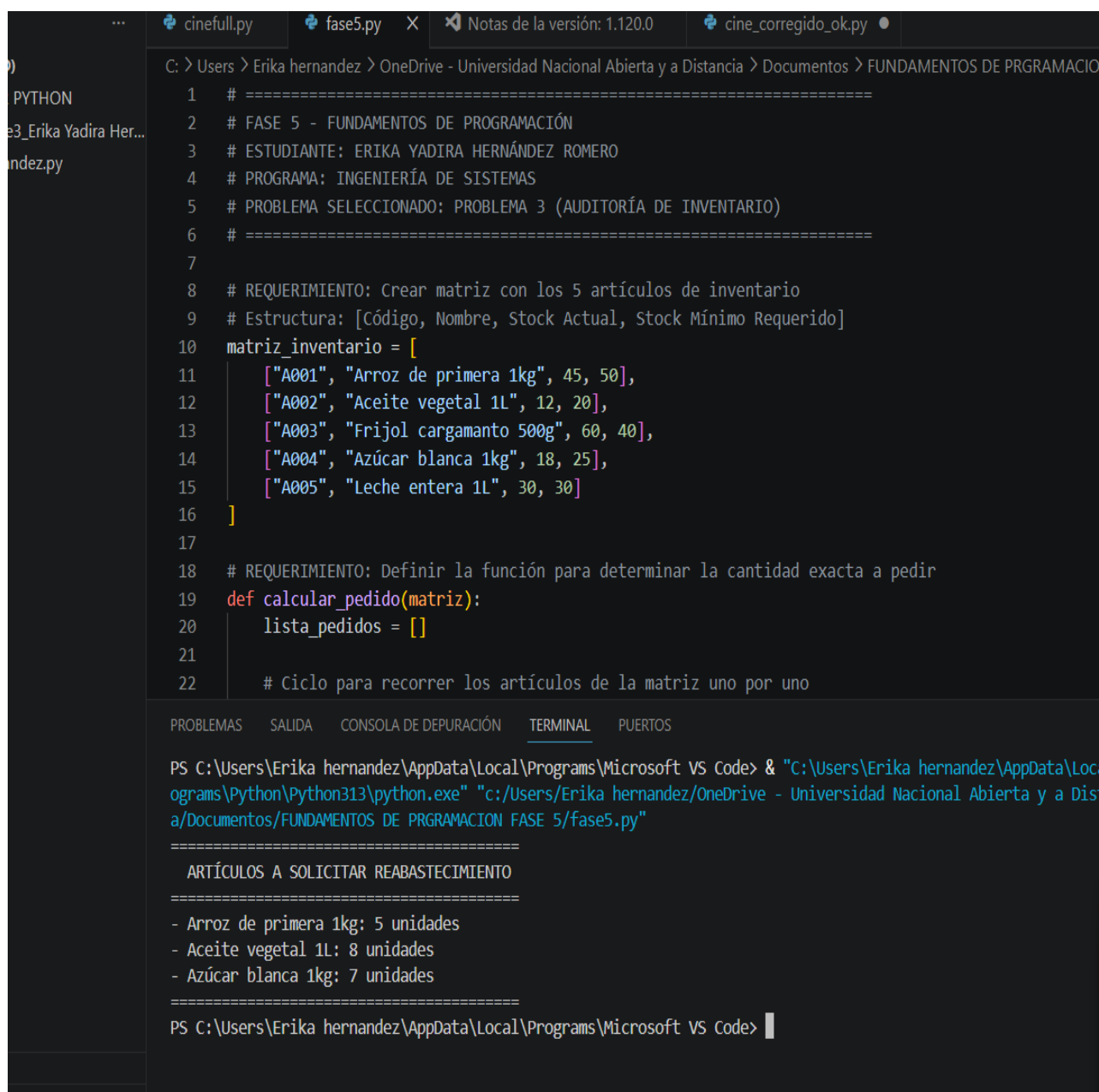
DESARROLLO DEL PROGRAMA EN PYTHON (Codigo)

Archivo: fase5.py

```
# =====  
# FASE 5 - FUNDAMENTOS DE PROGRAMACIÓN  
# ESTUDIANTE: ERIKA YADIRA HERNÁNDEZ ROMERO  
# PROGRAMA: INGENIERÍA DE SISTEMAS  
# PROBLEMA SELECCIONADO: PROBLEMA 3 (AUDITORÍA DE INVENTARIO)  
# =====  
  
# REQUERIMIENTO: Crear matriz con los 5 artículos de inventario  
# Estructura: [Código, Nombre, Stock Actual, Stock Mínimo Requerido]  
matriz_inventario = [  
    ["A001", "Arroz de primera 1kg", 45, 50],  
    ["A002", "Aceite vegetal 1L", 12, 20],  
    ["A003", "Frijol cargamanto 500g", 60, 40],  
    ["A004", "Azúcar blanca 1kg", 18, 25],  
    ["A005", "Leche entera 1L", 30, 30]  
]  
  
# REQUERIMIENTO: Definir la función para determinar la cantidad exacta a pedir  
def calcular_pedido(matriz):  
    lista_pedidos = []  
  
    # Ciclo para recorrer los artículos de la matriz uno por uno  
    for articulo in matriz:  
        codigo = articulo[0]  
        nombre = articulo[1]  
        stock_actual = articulo[2]  
        stock_minimo = articulo[3]  
  
        # Estructura de decisión: Validar condición de reabastecimiento  
        if stock_actual < stock_minimo:  
            # Fórmula: Cantidad exacta a pedir = Mínimo - Actual  
            cantidad_a_pedir = stock_minimo - stock_actual  
            lista_pedidos.append([nombre, cantidad_a_pedir])  
  
    return lista_pedidos
```

```
# =====  
# EJECUCIÓN DEL PROGRAMA PRINCIPAL  
# =====  
# Llamar a la función pasando la matriz como argumento  
resultados = calcular_pedido(matriz_inventario)  
  
# Mostrar los resultados formateados en la terminal  
print("=====  
print("  ARTÍCULOS A SOLICITAR REABASTECIMIENTO  ")  
print("=====  
for articulo in resultados:  
    print(f"- {articulo[0]}: {articulo[1]} unidades")  
print("=====
```

Evidencia del código ejecutado



The screenshot shows a Visual Studio Code editor with a Python script named `fase5.py` open. The script is a Python program for inventory management. It defines a matrix of inventory items and a function to calculate the quantity to order based on current stock and minimum required stock.

```
C:\> Users > Erika hernandez > OneDrive - Universidad Nacional Abierta y a Distancia > Documentos > FUNDAMENTOS DE PRGRAMACION
1  # =====
2  # FASE 5 - FUNDAMENTOS DE PROGRAMACIÓN
3  # ESTUDIANTE: ERIKA YADIRA HERNÁNDEZ ROMERO
4  # PROGRAMA: INGENIERÍA DE SISTEMAS
5  # PROBLEMA SELECCIONADO: PROBLEMA 3 (AUDITORÍA DE INVENTARIO)
6  # =====
7
8  # REQUERIMIENTO: Crear matriz con los 5 artículos de inventario
9  # Estructura: [Código, Nombre, Stock Actual, Stock Mínimo Requerido]
10 matriz_inventario = [
11     ["A001", "Arroz de primera 1kg", 45, 50],
12     ["A002", "Aceite vegetal 1L", 12, 20],
13     ["A003", "Frijol cargamanto 500g", 60, 40],
14     ["A004", "Azúcar blanca 1kg", 18, 25],
15     ["A005", "Leche entera 1L", 30, 30]
16 ]
17
18 # REQUERIMIENTO: Definir la función para determinar la cantidad exacta a pedir
19 def calcular_pedido(matriz):
20     lista_pedidos = []
21
22     # Ciclo para recorrer los artículos de la matriz uno por uno
```

The terminal output shows the execution of the script, displaying the inventory items and the calculated quantities to order.

```
PS C:\Users\Erika hernandez\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Erika hernandez\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Erika hernandez/OneDrive - Universidad Nacional Abierta y a Distancia/Documentos/FUNDAMENTOS DE PRGRAMACION FASE 5/fase5.py"
=====
ARTÍCULOS A SOLICITAR REABASTECIMIENTO
=====
- Arroz de primera 1kg: 5 unidades
- Aceite vegetal 1L: 8 unidades
- Azúcar blanca 1kg: 7 unidades
=====
PS C:\Users\Erika hernandez\AppData\Local\Programs\Microsoft VS Code>
```

EXPLICACION DEL PROGRAMA

(PROBLEMA 3)

1. Se crea una matriz llamada

```
2. matriz_inventario
```

con 5 artículos de la canasta básica.

2. cada fila de la matriz contiene:

- código de artículo
- nombre del artículo
- stock actual
- stock mínimo requerido

3. se implementa la función

```
cantidad_a_pedir = stock_minimo - stock_actual
```

para evaluar la lógica de negocio mediante la operación matemática de la resta.

4. la función recorre la matriz usando una estructura repetitiva

```
for
```

para evaluar producto por producto.

5. se validan dos condiciones:

- si el stock actual es menor al stock mínimo, la cantidad a pedir es la diferencia necesaria para reabastecer el artículo.
- Si el stock actual es suficiente (mayor o igual al mínimo) la cantidad a pedir es cero.

6. finalmente, se muestra en la pantalla de la terminal a ejecutar el código el informe consolidado con los nombres de los artículos y la cantidad exacta que debe ser solicitada.

```
PS C:\Users\Erika hernandez\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Erika hernandez\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Erika hernandez/OneDrive - Universidad Nacional Abierta y a Distancia/Documentos/FUNDAMENTOS DE PROGRAMACION FASE 5/fase5.py"
```

```
=====
```

```
ARTÍCULOS A SOLICITAR REABASTECIMIENTO
```

```
=====
```

- Arroz de primera 1kg: 5 unidades
- Aceite vegetal 1L: 8 unidades
- Azúcar blanca 1kg: 7 unidades

```
=====
```

```
PS C:\Users\Erika hernandez\AppData\Local\Programs\Microsoft VS Code> [ ]
```

```
fwd-i-search: _
```

Repositorio GitHub:

https://github.com/erikahernandez-unad/UNAD_Fundamentos_Programacion_Fase5

CONCLUSIONES

- A través del desarrollo de este programa en Python, logré comprender la utilidad práctica de las funciones y la condiciones para automatizar tareas operativas complejas como lo es el control minucioso de un inventario.
- Analice que la matriz no solo me permitió almacenar datos de manera estructurada, sino que que facilito que el programa corriera automáticamente cada producto para evaluar de forma inmediata su estado de disponibilidad del producto.
- Descubrí la importancia de la lógica en programación y la importancia de las herramientas digitales las cuales son eficientes para optimizar los procesos logísticos y evitar el desabastecimiento en entornos reales.

REFERENCIAS

- Microsoft. (en..). Configurar GitHub Copilot en Visual Studio Code. Visual Studio Code Documentación.
- Ceballos Sierra, F. J. (2015). C/C++. Curso de programación. (4a Edición).
- Deitel, P., & Deitel, H. (2016). Cómo programar en C/C++. Pearson Educación
- GitHub, Inc. (2025). Acerca de GitHub y Git. GitHub Docs.